

SKILL WEEK-11

1 . To connect different program modules, validate their interfaces, test end-to-end execution, handle errors, improve reliability, and refactor components.

Aim : To develop a modular C program where separate components communicate through well-defined interfaces, verify the complete execution flow, handle errors properly, and improve code reliability through refactoring.

Program :

```
GNU nano 7.2
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);

#endif
```

```
GNU nano 7.2
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}
```

```

GNU nano 7.2
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 10;
    int b = 5;

    printf("Module Integration Test\n");
    printf("-----\n");

    printf("A = %d\n", a);
    printf("B = %d\n\n", b);

    printf("Addition      : %d\n", add(a, b));
    printf("Subtraction    : %d\n", subtract(a, b));
    printf("Multiplication : %d\n", multiply(a, b));

    printf("\nEnd-to-end execution completed successfully.\n");

    return 0;
}

```

Output :

```

root@keerthana: ~/OSSP
root@keerthana:~# cd ~/OSSP
root@keerthana:~/OSSP# mkdir -p ~/OSSP
cd ~/OSSP
root@keerthana:~/OSSP# nano calculator.h
root@keerthana:~/OSSP# nano calculator.c
root@keerthana:~/OSSP# nano main_module.c
root@keerthana:~/OSSP# gcc -Wall -Wextra -c calculator.c -o calculator.o
root@keerthana:~/OSSP# gcc -Wall -Wextra -c main_module.c -o main_module.o
root@keerthana:~/OSSP# gcc calculator.o main_module.o -o module_test
root@keerthana:~/OSSP# ./module_test
Module Integration Test
-----

A = 10
B = 5
Addition      : 15
Subtraction    : 5
Multiplication : 50

End-to-end execution completed successfully.
root@keerthana:~/OSSP# ls -l *.c *.h *.o module_test
-rw-r--r-- 1 root root  169 Sep 27 14:20 calculator.c
-rw-r--r-- 1 root root  130 Sep 27 14:20 calculator.h
-rw-r--r-- 1 root root 1464 Sep 27 14:20 calculator.o
-rw-r--r-- 1 root root  468 Sep 27 14:20 main_module.c
-rw-r--r-- 1 root root 2416 Sep 27 14:20 main_module.o
-rwxr-xr-x 1 root root 16144 Sep 27 14:21 module_test
-rw-r--r-- 1 root root  767 Aug  3 05:13 task1.c
-rw-r--r-- 1 root root 2513 Sep 25 15:03 task_process_groups.c
-rw-r--r-- 1 root root 2234 Sep 23 05:15 task_resume_job.c
-rw-r--r-- 1 root root 2010 Sep 25 14:57 task_sigint_forward.c
-rw-r--r-- 1 root root 1550 Sep 25 14:52 task_signal_actions.c
-rw-r--r-- 1 root root 1616 Sep 23 05:03 task_signal_behavior.c

```

```

root@keerthana:~/OSSP# gcc calculator.o main_module.o -o module_test
root@keerthana:~/OSSP# ./module_test
Module Integration Test
-----

A = 10
B = 5
Addition      : 15
Subtraction   : 5
Multiplication : 50

End-to-end execution completed successfully.
root@keerthana:~/OSSP# ls -l *.c *.h *.o module_test
-rw-r--r-- 1 root root 169 Sep 27 14:20 calculator.c
-rw-r--r-- 1 root root 130 Sep 27 14:20 calculator.h
-rw-r--r-- 1 root root 1464 Sep 27 14:20 calculator.o
-rw-r--r-- 1 root root 468 Sep 27 14:20 main_module.c
-rw-r--r-- 1 root root 2416 Sep 27 14:20 main_module.o
-rwxr-xr-x 1 root root 16144 Sep 27 14:21 module_test
-rw-r--r-- 1 root root 767 Aug 3 05:13 task1.c
-rw-r--r-- 1 root root 2513 Sep 25 15:03 task_process_groups.c
-rw-r--r-- 1 root root 2234 Sep 23 05:15 task_resume_job.c
-rw-r--r-- 1 root root 2010 Sep 25 14:57 task_sigint_forward.c
-rw-r--r-- 1 root root 1550 Sep 25 14:52 task_signal_actions.c
-rw-r--r-- 1 root root 1616 Sep 23 05:03 task_signal_behavior.c
-rw-r--r-- 1 root root 2632 Sep 25 14:58 task_sigtstp.c
root@keerthana:~/OSSP# |

```

2 . To detect runtime errors, generate meaningful error messages, log failures, handle invalid syntax, recover gracefully, and test different error scenarios.

Aim : To implement a C program that detects and handles runtime and syntax errors, displays appropriate error messages, records failures in a log file, and continues execution safely after recoverable errors.

Program :

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

void log_error(const char *message)
{
    FILE *log = fopen("error.log", "a");

    if (log == NULL)
    {
        perror("Cannot open error log");
        return;
    }

    fprintf(log, "ERROR: %s\n", message);
    fclose(log);
}

int main()
{
    char input[100];
    int number;
    FILE *file;

    printf("Runtime Error and Syntax Handling\n");
    printf("-----\n");

    /* Test invalid file access */
    file = fopen("missing_file.txt", "r");

```

```

GNU nano 7.2
/* Test invalid file access */
file = fopen("missing_file.txt", "r");

if (file == NULL)
{
    printf("Error: Unable to open missing_file.txt\n");
    printf("Reason: %s\n", strerror(errno));

    log_error("Failed to open missing_file.txt");

    printf("Recovery: Continuing program execution...\n\n");
}
else
{
    fclose(file);
}

/* Test invalid numeric input */
printf("Enter an integer: ");

if (fgetc(input, sizeof(input), stdin) == NULL)
{
    printf("Error: Input could not be read.\n");
    log_error("Input reading failed");
    return 1;
}

char extra;

if (sscanf(input, "%d %c", &number, &extra) != 1)
{
    printf("Syntax Error: Invalid integer input.\n");
    printf("Recovery: Please enter a valid integer.\n");

    log_error("Invalid integer syntax");

    number = 0;
}

printf("Processed value: %d\n", number);

printf("\nProgram recovered successfully.\n");
printf("Testing completed.\n");

return 0;
}

```

Output :

```

root@keerthana: ~/OSSP
root@keerthana:~/OSSP# nano task_error_handling.c
root@keerthana:~/OSSP# gcc -Wall -Wextra task_error_handling.c -o task_error_handling
root@keerthana:~/OSSP# ./task_error_handling
Runtime Error and Syntax Handling
-----

Error: Unable to open missing_file.txt
Reason: No such file or directory
Recovery: Continuing program execution...

Enter an integer: 25
Processed value: 25

Program recovered successfully.
Testing completed.
root@keerthana:~/OSSP# ./task_error_handling
Runtime Error and Syntax Handling
-----

Error: Unable to open missing_file.txt
Reason: No such file or directory
Recovery: Continuing program execution...

Enter an integer: abc
Syntax Error: Invalid integer input.
Recovery: Please enter a valid integer.
Processed value: 0

Program recovered successfully.
Testing completed.
root@keerthana:~/OSSP#
root@keerthana:~/OSSP# cat error.log
ERROR: Failed to open missing_file.txt

```

```
Program recovered successfully.
Testing completed.
root@keerthana:~/OSSP# ./task_error_handling
Runtime Error and Syntax Handling
-----

Error: Unable to open missing_file.txt
Reason: No such file or directory
Recovery: Continuing program execution...

Enter an integer: abc
Syntax Error: Invalid integer input.
Recovery: Please enter a valid integer.
Processed value: 0

Program recovered successfully.
Testing completed.
root@keerthana:~/OSSP#
root@keerthana:~/OSSP# cat error.log
ERROR: Failed to open missing_file.txt
ERROR: Failed to open missing_file.txt
ERROR: Invalid integer syntax
root@keerthana:~/OSSP# |
```

3 . To run Valgrind, identify memory leaks and memory errors, fix the errors, release allocated resources, verify cleanup, and document the findings.

Aim : To analyze dynamic memory usage in a C program using Valgrind, detect memory leaks and invalid memory operations, correct the errors, and verify that all allocated memory is properly released.

Program :

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *name;
    char *message;

    printf("Valgrind Memory Test\n");
    printf("-----\n");

    name = malloc(50 * sizeof(char));
    message = malloc(100 * sizeof(char));

    if (name == NULL || message == NULL)
    {
        printf("Memory allocation failed.\n");
        free(name);
        free(message);
        return 1;
    }

    strcpy(name, "OSSP Student");
    strcpy(message, "Memory allocation successful.");

    printf("Name      : %s\n", name);
    printf("Message   : %s\n", message);

    /* Release allocated memory */
    free(name);
    free(message);

    name = NULL;
    message = NULL;

    printf("\nMemory released successfully.\n");

    return 0;
}
```

```

GNU nano 7.2
VALGRIND MEMORY ANALYSIS REPORT

Task:
Run Valgrind, identify leaks, fix memory errors,
release resources, verify cleanup, and document findings.

Program:
task_valgrind.c

Tool:
Valgrind

Initial Test:
Dynamic memory was allocated using malloc().

Problem Identified:
Memory that is allocated dynamically must be released
using free() before program termination.

Fix Applied:|
free() was used to release allocated memory.
Pointers were then set to NULL.

Final Verification:
Valgrind reported:
All heap blocks were freed -- no leaks are possible.

Error Summary:
0 errors from 0 contexts.

Result:
Memory was successfully allocated, used, released,
and verified using Valgrind.

```

Output :

```

root@keerthana: ~/OSSP
root@keerthana:~/OSSP# valgrind --version
valgrind-3.22.0
root@keerthana:~/OSSP# nano task_valgrind.c
root@keerthana:~/OSSP# gcc -Wall -Wextra -g task_valgrind.c -o task_valgrind
root@keerthana:~/OSSP# ./task_valgrind
Valgrind Memory Test
-----
Name      : OSSP Student
Message   : Memory allocation successful.

Memory released successfully.
root@keerthana:~/OSSP# valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind
==539== Memcheck, a memory error detector
==539== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==539== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==539== Command: ./task_valgrind
==539==
Valgrind Memory Test
-----
Name      : OSSP Student
Message   : Memory allocation successful.

Memory released successfully.
==539==
==539== HEAP SUMMARY:
==539==    in use at exit: 0 bytes in 0 blocks
==539==   total heap usage: 3 allocs, 3 frees, 1,174 bytes allocated
==539==
==539== All heap blocks were freed -- no leaks are possible
==539==
==539== For lists of detected and suppressed errors, rerun with: -s
==539== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@keerthana:~/OSSP# nano task_valgrind_leak.c

```

```

root@keerthana: ~/OSSP
Memory released successfully.
==539==
==539== HEAP SUMMARY:
==539==   in use at exit: 0 bytes in 0 blocks
==539==   total heap usage: 3 allocs, 3 frees, 1,174 bytes allocated
==539==
==539== All heap blocks were freed -- no leaks are possible
==539==
==539== For lists of detected and suppressed errors, rerun with: -s
==539== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@keerthana:~/OSSP# nano task_valgrind_leak.c
root@keerthana:~/OSSP# gcc -Wall -Wextra -g task_valgrind_leak.c -o task_valgrind_leak
root@keerthana:~/OSSP# ./task_valgrind_leak
Data: 1 2 3 4 5
root@keerthana:~/OSSP# valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind_leak
==549== Memcheck, a memory error detector
==549== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==549== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==549== Command: ./task_valgrind_leak
==549==
Data: 1 2 3 4 5
==549==
==549== HEAP SUMMARY:
==549==   in use at exit: 20 bytes in 1 blocks
==549==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==549==
==549== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==549==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==549==   by 0x1091BE: main (task_valgrind_leak.c:8)
==549==
==549== LEAK SUMMARY:
==549==   definitely lost: 20 bytes in 1 blocks
==549==   indirectly lost: 0 bytes in 0 blocks

```

```

root@keerthana: ~/OSSP
==549==   possibly lost: 0 bytes in 0 blocks
==549==   still reachable: 0 bytes in 0 blocks
==549==   suppressed: 0 bytes in 0 blocks
==549==
==549== For lists of detected and suppressed errors, rerun with: -s
==549== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
root@keerthana:~/OSSP# nano task_valgrind_leak.c
root@keerthana:~/OSSP# gcc -Wall -Wextra -g task_valgrind_leak.c -o task_valgrind_leak
root@keerthana:~/OSSP# valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind_leak
==556== Memcheck, a memory error detector
==556== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==556== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==556== Command: ./task_valgrind_leak
==556==
Data: 1 2 3 4 5
==556==
==556== HEAP SUMMARY:
==556==   in use at exit: 0 bytes in 0 blocks
==556==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==556==
==556== All heap blocks were freed -- no leaks are possible
==556==
==556== For lists of detected and suppressed errors, rerun with: -s
==556== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@keerthana:~/OSSP# nano valgrind_report.txt
root@keerthana:~/OSSP#

```

4 . To design test cases, create test scripts, automate their execution, verify outputs, compare expected and actual results, and generate test reports.

Aim : To implement an automated testing system for a C program that executes multiple test cases, checks the program output against expected results, and generates a test report.

Program :

```
GNU nano 7.2
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    int a = 10;
    int b = 5;

    printf("ADD: %d\n", add(a, b));
    printf("SUBTRACT: %d\n", subtract(a, b));
    printf("MULTIPLY: %d\n", multiply(a, b));

    return 0;
}
```

```

GNU nano 7.2
#!/bin/bash

echo "Automated Test Execution"
echo "=====

PASS=0
FAIL=0

echo ""
echo "Test 1: Addition"

EXPECTED="ADD: 15"
ACTUAL=$(./calculator_test | grep "ADD")

if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "PASS"
    PASS=$((PASS + 1))
else
    echo "FAIL"
    echo "Expected: $EXPECTED"
    echo "Actual   : $ACTUAL"
    FAIL=$((FAIL + 1))
fi

echo ""
echo "Test 2: Subtraction"

EXPECTED="SUBTRACT: 5"
ACTUAL=$(./calculator_test | grep "SUBTRACT")

if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "PASS"
    PASS=$((PASS + 1))
else
    echo "FAIL"
    echo "Expected: $EXPECTED"
    echo "Actual   : $ACTUAL"
    FAIL=$((FAIL + 1))
fi

echo ""
echo "Test 3: Multiplication"

```

```

EXPECTED="MULTIPLY: 50"
ACTUAL=$(./calculator_test | grep "MULTIPLY")

if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "PASS"
    PASS=$((PASS + 1))
else
    echo "FAIL"
    echo "Expected: $EXPECTED"
    echo "Actual   : $ACTUAL"
    FAIL=$((FAIL + 1))
fi

echo ""
echo "=====
echo "Test Summary"
echo "=====
echo "Passed: $PASS"
echo "Failed: $FAIL"

if [ $FAIL -eq 0 ]; then
    echo "Overall Result: PASS"
else
    echo "Overall Result: FAIL"
fi

```

```
GNU nano 7.2
Test 1: Addition
PASS

Test 2: Subtraction
PASS

Test 3: Multiplication
PASS|
```

Output :

```
root@keerthana: ~/OSSP
root@keerthana:~/OSSP# nano valgrind_report.txt
root@keerthana:~/OSSP# nano calculator_test.c
root@keerthana:~/OSSP# gcc -Wall -Wextra calculator_test.c -o calculator_test
root@keerthana:~/OSSP# ./calculator_test
ADD: 15
SUBTRACT: 5
MULTIPLY: 50
root@keerthana:~/OSSP# nano test_script.sh
root@keerthana:~/OSSP# chmod +x test_script.sh
root@keerthana:~/OSSP# ./test_script.sh
Automated Test Execution
=====

Test 1: Addition
PASS

Test 2: Subtraction
PASS

Test 3: Multiplication
PASS

=====
Test Summary
=====
Passed: 3
Failed: 0
Overall Result: PASS
root@keerthana:~/OSSP# ./test_script.sh > test_report.txt
root@keerthana:~/OSSP# ./test_script.sh > test_report.txt
root@keerthana:~/OSSP# cat test_report.txt
Automated Test Execution
=====
```

```
root@keerthana: ~/OSSP
Overall Result: PASS
root@keerthana:~/OSSP# ./test_script.sh > test_report.txt
root@keerthana:~/OSSP# ./test_script.sh > test_report.txt
root@keerthana:~/OSSP# cat test_report.txt
Automated Test Execution
=====

Test 1: Addition
PASS

Test 2: Subtraction
PASS

Test 3: Multiplication
PASS

=====
Test Summary
=====
Passed: 3
Failed: 0
Overall Result: PASS
root@keerthana:~/OSSP# nano expected_report.txt
root@keerthana:~/OSSP# diff expected_report.txt test_report.txt
root@keerthana:~/OSSP# ls -l calculator_test.c calculator_test test_script.sh test_report.txt expected_report.txt
-rwxr-xr-x 1 root root 16064 Sep 27 14:31 calculator_test
-rw-r--r-- 1 root root 355 Sep 27 14:31 calculator_test.c
-rw-r--r-- 1 root root 233 Sep 27 14:32 expected_report.txt
-rw-r--r-- 1 root root 233 Sep 27 14:32 test_report.txt
-rwxr-xr-x 1 root root 1187 Sep 27 14:32 test_script.sh
root@keerthana:~/OSSP#
```